

RELATIONAL SEPARATION LOGIC FOR COMPILER VERIFICATION

MPRI Internship Report

Gabriel DESFRENE

Supervised by Xavier LEROY and François POTTIER

General Context

Modern software relies on high-level languages and formal verification techniques, such as deductive verification, model checking, or static analysis, to establish correctness guarantees. Hardware executes machine code produced by compilers, which translate source programs through intermediate representations down to assembly. Consequently, source-level correctness guarantees remain meaningful only if compilation strictly preserves program semantics. Unverified compilation passes risk introducing bugs or invalidating source-level formal proofs.

Initial efforts in verified compilation established methodologies for proving semantic preservation. The CompCert compiler developed by Leroy [16] relies on simulation diagrams, establishing a correspondence between execution steps of the source and target programs while maintaining a state invariant. CakeML, a verified compiler for functional languages made by Tan et al. [25], combines simulation diagrams with equational reasoning to prove its correctness.

Beyond general-purpose compilers, other verified compiler projects focus on specialized application domains. For instance, Jinja [12] targets object-oriented programs compiled to the Java Virtual Machine, whereas Jasmin [2] verifies a dedicated compiler for high-performance cryptographic software.

Existing verification paradigms nevertheless exhibit limitations. On one hand, equational theories are often incomplete, especially when programs differ in their memory representations. On the other hand, traditional simulation diagrams grow complex when program control flow structures diverge significantly, for instance after nested loop exchanges.

Recent advances in formal verification address these shortcomings. Modern coinductive simulation techniques, such as those developed by Cho et al. [4] in the *Stuttering for Free* framework or implemented by Xia et al. [27] within the *Interaction Trees* library, simplify the reasoning about program pairs whose execution steps do not correspond one-to-one.

In parallel, *relational separation logics* have emerged as a technique for local reasoning across dual executions, allowing memory resources to be tracked independently or partitioned between two programs. For instance, the *Simuliris* framework by Gäher et al. [6] uses a relational separation logic to verify source-to-source program transformations. Recent work on the *Tail Modulo Cons* optimization by Allain et al. [1] employs relational separation logic to validate the soundness of the optimization within an idealized framework.

Research Problem

This internship investigates whether recent developments in simulation relations and separation logic can be integrated into a unified relational separation logic framework. To our knowledge, no existing verified compiler incorporates a relational separation logic to prove the soundness of its optimization passes. Consequently, establishing such a framework would enable compositional compiler verification with improved scalability and fewer language restrictions. In short, this approach relaxes constraints present in existing systems, such as the determinism requirements imposed by CompCert and CakeML.

Contributions

We developed a logical framework addressing the objectives of this internship. At its core, the framework relies on a coinductive simulation relation capable of matching execution steps of programs with different control flow.

42 The simulation used is skewed: it permits exploiting undefined behaviors in the source program to enable further
43 target-level optimizations. This simulation serves as the foundation of our verification framework, defining how
44 source and target execution steps are related.

45 Building upon this simulation relation, we defined a relational separation logic that enables local reasoning
46 over fragments of source and target heaps. The logic is parameterized by the operational semantics of the two
47 languages under comparison. Our framework uses this logical layer to express and prove relational specifications
48 between program pairs.

49 We instantiated this generalized framework with a low-level language that mirrors the RTL language (Register
50 Transfer Language), commonly used in compiler backends. We defined the small-step operational semantics for
51 our version of RTL, with support for arithmetic operations, dynamic memory allocation and deallocation, and
52 function calls.

53 Finally, we established a system of reasoning rules for RTL programs. This proof system comprises two distinct
54 categories of rules. First, we proved the soundness of asynchronous rules, which deal with unilateral execution
55 steps taken independently by either the source or the target. In particular, these rules enable target optimizations
56 to exploit undefined behaviors in the source program. Second, we derived synchronous rules, which deal with
57 joint execution steps when both source and target programs execute matching instructions at the same time.

58 **Arguments Supporting Its Validity**

59 We evaluated our framework by proving the correctness of three program transformations. The first one is a
60 branch tunneling pass that eliminates redundant control-flow jumps by collapsing chains of intermediate nodes.
61 The second one is an instruction-level optimization pass that removes statically known redundant operations.
62 Finally, we used our logic to establish the soundness of localized transformations, such as hoisting a memory
63 read instruction out of a loop body.

64 Beyond these practical case studies, we established metatheoretic properties confirming framework soundness.
65 We proved that the underlying simulation relation is sound, guaranteeing that it relates only programs exhibiting
66 related behaviors. Additionally, we proved our simulation formulation equivalent to established simulation
67 definitions used in the literature [1, 16, 6].

68 The entire framework has been mechanized in the Rocq proof assistant [26]. All definitions and theorems are
69 machine checked, relying on the coinduction Rocq library [20] and the Iris/stdpp [10, 9] data structures. A
70 subset of the formalization relies on two standard classical axioms, specifically the Law of Excluded Middle and
71 Classical Choice.

72 **Future Work**

73 This work provides a first step for several research perspectives. While current case studies verify control flow
74 transformations, future effort will target data flow optimizations, such as constant propagation and common
75 subexpression elimination. Such passes will test the logical framework when memory states are altered by
76 eliminated loads, stores or allocations.

77 A second direction involves supporting multi-language verification. Our framework is parameterized to relate
78 distinct source and target languages across translation passes, but the present formalization operates on a single
79 intermediate language. Future effort will test the limits of our framework when translating programs from one
80 language to another.

81 Finally, we aim to increase the expressiveness of the underlying languages. While our logic already accommodates
82 nondeterminism, it is not present in our current semantics of RTL. We plan to extend our language semantics
83 with it to work toward concurrent execution features. Additionally, extending the framework from imperative
84 languages to functional paradigms represents another potential direction for future work.

1 Verified Compilation Background

1.1 Small-Step Operational Semantics

Giving a formal semantics to a programming language provides rigorous mathematical meaning to its programs. Small-step operational semantics models computation as a sequence of elementary transitions. It describes how the internal state evolves under primitive operations rather than evaluating compound statements in a single step. This style directly captures non-termination as infinite transition sequences and allows intermediate execution states to be inspected.

Formally, a small-step operational semantics is defined as a transition system $(S, \rightarrow)$, where S represents the set of program states and $\rightarrow \subseteq S \times S$ denotes the binary reduction relation. In our framework, a state is a pair (s, m) of an internal program state s and a memory m . To allow transitions to inspect program code and function declarations, the reduction relation is parameterized by the global program P . We write $P \vdash (s, m) \rightarrow (s', m')$ to indicate that configuration (s, m) reduces in one step to (s', m') under program P , and denote by $\rightarrow^*$ its reflexive transitive closure. We do not assume that this step relation is deterministic.

To characterize execution endpoints, our framework incorporates a partial function `is_value`, which maps internal states to optional return values. A state (s, m) is considered final when `is_value(s)` yields a value v . For such final states, no further reduction steps are possible under any memory in program P , signifying the formal end of execution.

The predicate `can_progressP(s, m)` indicates that (s, m) can perform a transition in P , meaning that $P \vdash (s, m) \rightarrow (s', m')$ for some s' and m' . Conversely, a configuration (s, m) is *stuck* in program P if it is not a final value state and cannot perform any step in P . Stuck states represent execution failures where the semantics is undefined. For instance, in an imperative language, a state becomes stuck when dereferencing an unallocated pointer address, dividing by zero, or branching to an invalid control-flow node identifier.

1.2 Program Behaviors

Equipped with a small-step operational semantics, we can formally classify the observable behaviors of a program state. Because our framework handles non-deterministic languages, a single initial state may give rise to multiple execution paths, and thus exhibit multiple behaviors. For a fixed program P , we classify execution paths into three distinct behaviors:

- A state *terminates* with value v and final memory m if execution reaches a final state (s', m) such that `is_value(s') = Some(v)`.
- A state *diverges* if there exists an infinite reduction sequence starting from it. Note that we do not restrict divergence to strongly diverging states, where every execution path is infinite.
- A state exhibits *undefined behavior* if execution reaches a stuck state.

In the following, we write $b \in s$ to denote that behavior b is exhibited by state s . We extend this notion to a complete program P by considering the behaviors of its initial state. Having established these behavior categories, we prove that every state admits an outcome:

Lemma 1. For any program P , every state s admits at least one behavior.

1.3 Program Refinement

The objective of a verified compiler is to guarantee that the correctness properties of a source program remain valid for the compiled executable. To ensure that compilation does not introduce bugs or invalidate source-level guarantees, the compiler must preserve semantic behavior. The presentation below follows the semantic preservation framework introduced by Leroy [16].

A program specification describes the acceptable observable outcomes of an execution. Formally, a specification `Spec` is a predicate over behaviors, written `Spec(b)`. A meaningful specification must rule out any undefined behavior: $\forall b, \text{Spec}(b) \implies b \neq \text{Undefined}$. We say that a program S satisfies a specification `Spec`, written $S \models \text{Spec}$, if every behavior exhibited by S satisfies the predicate: $\forall b, b \in S \implies \text{Spec}(b)$.

130 Given a source program S and its compiled target executable T , the goal of compiler verification is to guarantee
 131 the *specification preservation* property:

$$S \models \text{Spec} \implies T \models \text{Spec} \quad (1)$$

132 Proving specification preservation directly for each individual specification is impractical, as the compiler verifi-
 133 cation proof should remain independent of specific application requirements. To decouple compiler verification
 134 from specific application requirements, semantic preservation is established by proving a *behavioral refinement*
 135 between the target program T and source program S :

$$T \sqsubseteq S := \forall b, b \in T \implies b \in S \quad (2)$$

136 If this property holds, specification preservation (1) follows immediately. This property allows the compiler to
 137 eliminate behaviors of the source program, such as resolving non-deterministic constructs by selecting a single
 138 outcome.

139 However, direct behavior refinement (2) is overly restrictive for optimizing compilers. If a source program only
 140 exhibits undefined behavior, an optimizing compiler may eliminate an erroneous code path or transform it into
 141 a benign execution for the target. Because meaningful specifications already exclude undefined behaviors, to
 142 guarantee the *specification preservation* property (1), it suffices to establish a *skewed* behavioral inclusion.

143 We first define a preorder relation on behaviors:

$$\begin{array}{c} \frac{}{\text{Terminating}(v, m) \leq_{\text{beh}} \text{Terminating}(v, m)} \text{Beh-Term} \\ \frac{}{\text{Diverging} \leq_{\text{beh}} \text{Diverging}} \text{Beh-Div} \quad \frac{}{b_t \leq_{\text{beh}} \text{Undefined}} \text{Beh-Undef} \end{array}$$

144 Under this preorder, terminating and diverging behaviors must be preserved identically, whereas an undefined
 145 behavior in the source can be refined by any target behavior b_t . We then define our notion of *skewed behavior*
 146 *refinement*. We say that a target state t refines a source state s , written $t \leq s$, when:

$$t \leq s := \forall b_t, b_t \in t \implies \exists b_s, b_s \in s \wedge b_t \leq_{\text{beh}} b_s$$

147 This relation is lifted to programs ($T \leq S$) by relating their initial states. When a program refines another, and
 148 because specifications rule out undefined behaviors, we obtain the *specification preservation* property (1).

149 1.4 Simulation Relations

150 While the refinement relation $t \sqsubseteq s$ is formulated globally in terms of whole-program behaviors, proving it
 151 directly over entire execution traces is impractical. Compilers perform local transformations whose correctness
 152 is best established step by step. To bridge this gap, verified compilation relies on *simulation relations*, which
 153 establish an invariant relating intermediate operational states of the source and target programs. This can be
 154 done in two flavors:

155 **Backward simulation:** each step taken by the target program must be matched by steps in the source program.
 156 Because it maps every target behavior back to a valid source execution, a backward simulation directly
 157 establishes the refinement property $t \sqsubseteq s$.

158 **Forward simulation:** each step of the source program is matched by corresponding steps in the target program.
 159 Forward simulations are more convenient to establish as compiler passes transform source constructs into
 160 target code, making inductive invariant proofs follow the structure of source reductions directly.

161 1.5 Coinductive Predicates

162 Proving a simulation relation requires relating operational execution steps of the source and target programs.
 163 When programs terminate, their execution consists of a finite sequence of reductions ending in a final state. Such
 164 finite behaviors are naturally formalized using *inductive* predicates. However, many programs are designed to run
 165 indefinitely. For non-terminating executions, the reduction sequence is infinite and never reaches a terminal base
 166 case. Establishing the simulation inductively is impossible, as it would require an infinite derivation tree, which

inductive definitions forbid. To reason about infinite execution traces and divergence, we rely on *coinductive* predicates.

The presentation adopted below, which defines coinduction in terms of infinite derivation trees, is a simplification adapted to our setting. For a proper definition of coinduction and coinductive predicates, we refer the reader to Milner [18], Giménez [7], and Pous [19]. Throughout this report, we define inductive and coinductive relations through sets of inference rules:

- *Inductive rules* are denoted with a single horizontal bar:

$$\frac{H_1 \quad \dots \quad H_n}{\mathcal{I}}$$

Here, H_i are the premises and $\mathcal{I}$ is the inductive conclusion. A judgment holds inductively if and only if it can be established through a *finite derivation tree*, starting from base cases.

- *Coinductive rules* are denoted with a double horizontal bar:

$$\frac{H_1 \quad \dots \quad H_n}{\mathcal{C}}$$

Here, H_i are the premises and $\mathcal{C}$ is the coinductive conclusion. They generalize *inductive* rules to judgments admitting a *potentially infinite derivation tree*.

For instance, defining finite lists of natural numbers uses induction, requiring every list to terminate at nil. In contrast, infinite streams are defined by coinduction, allowing endless constructor applications without base cases:

$$\frac{}{\text{nil} \in \text{List}(\mathbb{N})} \quad \frac{n \in \mathbb{N} \quad l \in \text{List}(\mathbb{N})}{\text{cons}(n, l) \in \text{List}(\mathbb{N})} \quad \frac{n \in \mathbb{N} \quad s \in \text{Stream}(\mathbb{N})}{\text{scons}(n, s) \in \text{Stream}(\mathbb{N})}$$

These differences in derivation tree restrictions translate directly into distinct proof principles:

- When proving a property over an inductive structure, one reasons by induction. We proceed by case analysis on the constructors of the inductive term (nil and cons for lists). The induction hypothesis may only be invoked on *strictly smaller subterms*. This finite descent ensures that the proof eventually terminates at a base case.
- Conversely, when proving that a property holds coinductively, one constructs an infinite derivation tree by coinduction. The coinductive hypothesis refers to the entire conclusion, but it can only be invoked *underneath a constructor* (scons for streams). This condition, known as *productivity*, prevents using the coinductive hypothesis to build circular proof trees.

A detailed example illustrating coinductive reasoning is provided in section A. The Rocq theorem prover [26] supports coinductive predicates natively. In our formalization, we use the coinduction library developed by Pous [20] to facilitate reasoning with coinductive simulation invariants.

1.6 Simulation Schemes & Stuttering

Equipped with coinductive predicates to handle potentially infinite executions, we can define a simulation relation to establish program refinements. Given a target program P_t and a source program P_s , a backward simulation relates a target state t to a source state s , ensuring that every execution step taken by the target program can be matched by the source program.

For terminating executions, the simulation requires that final states yield identical observable values. Formally, if both states reach terminal configurations, they are related by the following base case:

$$\frac{\text{is_value}(t) = \text{Some}(v) \quad \text{is_value}(s) = \text{Some}(v)}{t \sim_l s} \quad (3)$$

For non-final configurations, the relation must ensure that the target state is capable of progressing and that every transition it performs corresponds to an execution step in the source program. Because executions may be non-terminating, this condition is expressed coinductively:

$$\frac{\text{can_progress}_{P_t}(t) \quad \forall t', P_t \vdash t \rightarrow t' \implies \exists s', P_s \vdash s \rightarrow s' \wedge t' \sim_l s'}{t \sim_l s} \quad (4)$$

204 The relation $\sim_l$ defined by eqs. (3) and (4) is a *lock-step* backward simulation: the target and source programs
 205 advance at the exact same pace. Under non-determinism, the target and source states are treated asymmetrically:

- 206 • The target program exhibits *demonic* non-determinism, requiring the simulation to hold for all reachable
 207 successor states t' .
- 208 • The source program exhibits *angelic* non-determinism, meaning that the proof only needs to exhibit the
 209 existence of a suitable matching successor state s' .

210 This asymmetry is precisely what establishes the behavior inclusion. We must account for all non-deterministic
 211 choices of the target and match each of them with an admissible path in the source.

212 While lock-step simulations suffice for simple transformations that preserve step counts, realistic compiler passes
 213 break this correspondence. An optimization pass such as branch tunneling removes redundant jump operations,
 214 causing the source program to perform steps that have no counterpart in the target. Conversely, lowering
 215 transformations decompose a single high-level source instruction into multiple low-level target instructions,
 216 requiring the target to execute several steps before the source advances.

217 To accommodate passes that alter step counts, one must allow *asynchronous* execution steps, where one program
 218 takes a step while the other remains stationary. Such a transition is called a *stuttering* step. A first coinductive
 219 formulation might introduce the following stuttering rules:

$$\frac{\text{can_progress}_{P_t}(t) \quad \forall t', P_t \vdash t \rightarrow t' \implies t' \sim s}{t \sim s} \quad \frac{P_s \vdash s \rightarrow s' \quad t \sim s'}{t \sim s}$$

220 However, adding these rules to the lock-step simulation $\sim_l$ invalidates the refinement between the target and the
 221 source. Without restriction, a target program could perform infinitely many stuttering steps without the source
 222 program ever progressing. Under such a definition, a trivially diverging target program could be related to any
 223 source state.

224 To guarantee that the target refines the source, stuttering must remain finite. This is achieved by equipping
 225 the simulation with a well-founded measure $(W, \prec)$. The relation is indexed by a measure $i \in W$, written
 226 $t \sim_e^i s$. Whenever a stuttering step occurs, the index strictly decreases according to $\prec$. When both programs step
 227 simultaneously, progress is made and the measure may be freely reset.

228 Because stuttering is explicitly represented, this relation is called the *explicit simulation* [4]. The inference rules
 229 defining it are given in fig. 1. Since $(W, \prec)$ is well-founded, an infinite sequence of stuttering steps is impossible.
 230 This guarantees that the explicit simulation implies behavior refinement.

$$\begin{array}{c} \frac{\text{is_value}(t) = \text{is_value}(s) = \text{Some}(v)}{t \sim_e^i s} \text{ERELATED} \quad \frac{P_s \vdash s \rightarrow s' \quad i' \prec i \quad t \sim_e^{i'} s'}{t \sim_e^i s} \text{ESOURCESTEPS} \\[10pt] \frac{\text{can_progress}_{P_t}(t) \quad \forall t', P_t \vdash t \rightarrow t' \implies \exists i', i' \prec i \wedge t' \sim_e^{i'} s}{t \sim_e^i s} \text{ETARGETSTEPS} \\[10pt] \frac{\text{can_progress}_{P_t}(t) \quad \forall t', P_t \vdash t \rightarrow t' \implies \exists s', \exists i', P_s \vdash s \rightarrow s' \wedge t' \sim_e^{i'} s'}{t \sim_e^i s} \text{EBOTHSTEPS} \end{array}$$

Figure 1: Derivation rules of the *explicit simulation*.

231 2 The Free Simulation

232 While explicit simulations effectively prevent infinite stuttering, they impose a substantial proof burden. Every
 233 stuttering transition requires exhibiting a strictly decreasing measure in a well-founded order. Constructing
 234 such a measure can be difficult in practice, particularly when compiler optimizations rearrange loop structures.
 235 In such scenarios, finding a global metric that strictly decreases on every asynchronous step while remaining
 236 invariant across complex control transfers is often impractical.

237 To overcome this rigidity, we adapt the *Stuttering for Free* framework introduced by Cho et al. [4]. We extend
 238 their foundational technique to accommodate skewed simulation relations. We have formalized and verified all
 239 underlying theoretical results within our Rocq development.

240 2.1 Skewed Implicit Simulation

241 A possible approach to eliminating explicit stuttering measures is to integrate inductive and coinductive reasoning
 242 into a single relation. In this formulation, finite sequences of asynchronous execution steps are justified inductively,
 243 whereas mutual progress steps are justified coinductively. This family of relations is called *implicit simulations*.
 244 We define a skewed implicit simulation relation, denoted $t \lesssim s \{\Phi\}$, whose inference rules are presented in fig. 2.
 245 To combine inductive and coinductive reasoning within the same definition, the relation $t \lesssim s \{\Phi\}$ is defined
 246 as the greatest fixed point¹ of an inductive operator. The rules IRELATED, ISOURCESTUCK, ISOURCESTEPS, and
 247 ITARGETSTEPS serve as inductive constructors, while the rule IBOTHSTEPS acts as the coinductive constructor.

$$\begin{array}{c}
 \frac{\text{both_final}_\Phi(t, s)}{t \lesssim s \{\Phi\}} \text{ IRELATED} \qquad \frac{\text{stuck}_{P_s}(s)}{t \lesssim s \{\Phi\}} \text{ ISOURCESTUCK} \\
 \\
 \frac{P_s \vdash s \rightarrow s' \quad t \lesssim s' \{\Phi\}}{t \lesssim s \{\Phi\}} \text{ ISOURCESTEPS} \\
 \\
 \frac{\text{can_progress}_{P_t}(t) \quad \forall t', P_t \vdash t \rightarrow t' \implies t' \lesssim s \{\Phi\}}{t \lesssim s \{\Phi\}} \text{ ITARGETSTEPS} \\
 \\
 \frac{\text{can_progress}_{P_t}(t) \quad \forall t', P_t \vdash t \rightarrow t' \implies \exists s', P_s \vdash s \rightarrow s' \wedge t' \lesssim s' \{\Phi\}}{t \lesssim s \{\Phi\}} \text{ IBOTHSTEPS}
 \end{array}$$

Figure 2: Derivation rules of the *skewed implicit simulation*.

248 The rule IRELATED characterizes terminating executions. Instead of demanding syntactic equality between
 249 terminal values, it is parameterized by a postcondition Φ that relates the final values and the concluding memory
 250 states of the target and source programs. The predicate `both_final` is defined as follows:

$$\begin{aligned}
 &\Phi : \text{val}_t \rightarrow \text{val}_s \rightarrow \text{mem} \rightarrow \text{mem} \rightarrow \text{Prop} \\
 &\text{both_final}_\Phi((t, m_t), (s, m_s)) := \exists v_t v_s, \text{is_value}(t) = \text{Some}(v_t) \wedge \text{is_value}(s) = \text{Some}(v_s) \wedge \\
 &\quad \Phi(v_t, v_s, m_t, m_s)
 \end{aligned}$$

251 The assertion `both_finalΦ(t, s)` holds when both target and source configurations have reached final states,
 252 producing values v_t, v_s and memories m_t, m_s that satisfy Φ . This parameterization enables the simulation to
 253 enforce relational invariants on memories at the end of execution.

254 The rule ISOURCESTUCK introduces more asymmetry to the relation. When the source program encounters
 255 undefined behavior it enters a stuck state in a non-final configuration. In this case, ISOURCESTUCK immediately
 256 validates the simulation, leaving the target program free to perform arbitrary transitions or diverge. This
 257 asymmetry allows compiler transformations that rely on the absence of undefined behavior in well-formed source
 258 programs to eliminate redundant runtime checks.

259 Stuttering is soundly handled without an explicit decreasing measure. Because ISOURCESTEPS and ITARGETSTEPS
 260 are inductive constructors, they permit only finitely many consecutive asynchronous transitions. The coinductive
 261 constructor IBOTHSTEPS requires both programs to execute a synchronized step, to remove the inductive proof
 262 obligation and enable unbounded mutual progress.

263 2.2 Step-Indexed Free Simulation

264 While the implicit simulation eliminates manual stuttering measures, it remains restrictive: the coinductive rule
 265 IBOTHSTEPS demands that both programs step at the exact moment when coinductive reasoning is resumed.

¹The relationship between greatest fixed points and coinduction follows from the Knaster–Tarski theorem; see [19, 21] for comprehensive treatments.

266 Removing this lock-step synchronization requirement leads to smaller invariants and more direct verification
 267 proofs [4]. This decoupling is the core idea developed in the *Stuttering for Free* framework.

268 The *free simulation*, denoted $t \lesssim_i s \{\Phi\}$, overcomes this limitation by decoupling synchronous execution steps
 269 from coinductive progress. The relation is parameterized by two well-founded index domains, $(J, \prec_J)$ for the
 270 target side and $(I, \prec_I)$ for the source side. These domains track the progress accumulated by each side since the
 271 previous coinductive step. The formal inference rules of the skewed free simulation are presented in fig. 3. Like
 272 the implicit simulation, it accommodates arbitrary postconditions Φ and uses the rule FSOURCESTUCK to exploit
 273 any undefined behavior in the source.

$$\begin{array}{c}
 \frac{\text{both_final}_\Phi(t, s)}{t \lesssim_i s \{\Phi\}} \text{FRELATED} \qquad \frac{\text{stuck}_{P_s}(s)}{t \lesssim_i s \{\Phi\}} \text{FSOURCESTUCK} \\
 \\
 \frac{P_s \vdash s \rightarrow s' \quad t \lesssim_{i'} s' \{\Phi\}}{t \lesssim_i s \{\Phi\}} \text{FSOURCESTEPS} \\
 \\
 \frac{\text{can_progress}_{P_t}(t) \quad \forall t', P_t \vdash t \rightarrow t' \implies \exists j', t' \lesssim_{j'} s \{\Phi\}}{t \lesssim_i s \{\Phi\}} \text{FTARGETSTEPS} \\
 \\
 \frac{j' \prec_J j \quad i' \prec_I i \quad t \lesssim_{i'} s \{\Phi\}}{t \lesssim_i s \{\Phi\}} \text{FPROGRESS}
 \end{array}$$

Figure 3: Derivation rules of the *skewed free simulation*.

274 The design of the free simulation relies on a fundamental shift in perspective regarding indices. In an explicit
 275 simulation, an index represents an upper bound on *future* stuttering steps. In the free simulation, indices record
 276 *past* progress accumulated since the previous coinductive step.

277 The source and target programs take transitions asynchronously via the inductive rules FTARGETSTEPS and
 278 FSOURCESTEPS. Whenever a program takes a step, its corresponding step index is updated (j' or i'). We note that
 279 j' and i' are not required to be immediate successors of j and i . Instead the user is free to select any new index,
 280 smaller or larger.

281 This flexibility does not compromise soundness. Indeed, the only rule that enables coinductive reasoning is
 282 FPROGRESS. This rule requires both indices to strictly decrease, from j down to $j' \prec_J j$ and from i down to $i' \prec_I i$.
 283 Because the relations $\prec_J$ and $\prec_I$ are well-founded, establishing a simulation between non-terminating executions
 284 demands that FPROGRESS be applied infinitely often, which in turn requires indices to be repeatedly increased
 285 through FTARGETSTEPS and FSOURCESTEPS. This structure decouples stepping from coinductive synchronization,
 286 allowing multiple asynchronous transitions on either side to be matched flexibly.

287 Moreover, performing coinductive reasoning over the free simulation is equivalent to the following principle: to
 288 prove $t \lesssim_i s \{\Phi\}$, one may assume the coinductive hypothesis:

$$\forall j' \succ_J j, \forall i' \succ_I i, \quad t \lesssim_{i'} s \{\Phi\}.$$

289 Intuitively, this principle allows one to establish that two programs simulate each other by assuming that the
 290 simulation relation already holds for the current states if they are reached after progress on both sides. A concrete
 291 example illustrating step-indexed reasoning in the free simulation is presented in section B.

292 2.3 Properties

293 Proving a free simulation for smaller indices implies that it holds for larger indices. This monotonicity property
 294 is valid not only as an external lemma, but also within coinductive proofs. This property is captured by index
 295 monotonicity:

296 **Lemma 2** (Monotonicity of Free Simulation). If $t \lesssim_i s \{\Phi\}$ holds, then for all indices $j \prec_J j'$ and $i \prec_I i'$, and
 297 any weaker postcondition $\Phi \implies \Phi'$, the simulation $t \lesssim_{i'} s \{\Phi'\}$ holds.

298 Next, the free simulation is sound with respect to the skewed behavioral refinement:

Theorem 1 (Soundness of Free Simulation). Let P_t and P_s be programs. Let t be the initial state of P_t and s be the initial state of P_s . If $t \lesssim_i s \{=\}$ holds for some indices j and i , then $P_t \leq P_s$.

In addition to soundness, we established the formal equivalence among the three simulation frameworks discussed: the free simulation (defined in fig. 3), the implicit simulation (defined in fig. 2), and a skewed version of the explicit simulation (defined in fig. 1 augmented with a stuck-source rule). This equivalence is established by chaining cyclic implications:

$$\text{Explicit Simulation} \xrightarrow{\textcircled{1}} \text{Implicit Simulation} \xrightarrow{\textcircled{2}} \text{Free Simulation} \xrightarrow{\textcircled{3}} \text{Explicit Simulation}$$

The implications $\textcircled{1}$ and $\textcircled{2}$ proceed directly by simulation coinduction. The last implication $\textcircled{3}$ is the most intricate: it requires synthesizing a well-founded stuttering measure from the derivation tree of the free simulation. A direct consequence of this equivalence is the index irrelevance theorem:

Theorem 2 (Index Irrelevance). Let $(J, \prec_J)$ and $(I, \prec_I)$ be well-founded relations without isolated elements². If there exist indices $j \in J$ and $i \in I$ such that $t \lesssim_i s \{\Phi\}$, then for any other well-founded structures $(J', \prec_{J'})$ and $(I', \prec_{I'})$, and any indices $j' \in J'$ and $i' \in I'$, the relation $t \lesssim_{i'} s \{\Phi\}$ holds.

Our statement refines the index irrelevance theorem formulated by Cho et al. [4]. Their original formulation, phrased in terms of ordinals, implicitly assumes that every element participates in the order. However, isolated elements could cause FPROGRESS transitions to be uninhabited, preventing the equivalence. This distinction was identified during our Rocq formalization of the metatheory of the free simulation.

This theorem implies that any free simulation derivation can be carried out using the two-element boolean order $\mathbb{B} = \{\text{false} \prec \text{true}\}$:

$$(\exists j \in J, i \in I, t \lesssim_i s \{\Phi\}) \iff (\exists j, i \in \mathbb{B}, t \lesssim_i s \{\Phi\}).$$

Under the boolean order, false indicates that a program has not yet taken a step, whereas true signals that progress has occurred. This means that, in theory, the free simulation proofs do not strictly require tracking the exact step counts of individual programs, but only whether progress has occurred since the last coinductive step. Nevertheless, in the remainder of our work, we adopt the natural numbers $(\mathbb{N}, <)$ as our index domain. Indeed, natural number indices correspond directly to the number of steps executed during symbolic evaluation, and they provide some intuition that facilitates the interactive verification in Rocq.

3 The Register Transfer Language (RTL)

To evaluate our framework on realistic compiler intermediate representations, we formalize a low-level imperative language based on the Register Transfer Language (RTL) of CompCert [16]. This language has two kinds of storage: local register banks attached to execution states, which act as local variables, and a global shared heap memory.

3.1 Values, Registers, and Memory

Computations in RTL manipulate first-order values. There are four kinds of values, where loc represents allocated heap locations:

$$v \in \text{val} ::= \text{VInt}(z) \mid \text{VBool}(b) \mid \text{VPtr}(l) \mid \text{VUndef} \\ \text{with } z \in \mathbb{Z}, b \in \{\text{true}, \text{false}\}, \text{ and } l \in \text{loc}$$

VUndef represents an undefined value, used to model uninitialized data. It can be copied between registers, stored into memory cells, or passed as function arguments. However, attempting to execute an instruction that requires a specific kind of value, such as an arithmetic calculation, will cause the execution to get stuck.

To relate source and target values, we define a similarity predicate $v_t \bowtie_I v_s$. This predicate is parameterized by a binary relation on locations, $I \subseteq \text{loc} \times \text{loc}$. This relation will be made precise later in section 4.2. The deduction rules of the value similarity predicate are given in fig. 4. We note that the rule VAL-UNDEF is asymmetric: whenever the source program produces an undefined value VUndef, it can be related to any value v_t produced by the target program. This asymmetry allows the compiler to replace undefined source computations with concrete target operations.

²An element is isolated if it has no predecessors or successors in the relation.

$$\begin{array}{c}
\frac{z_t = z_s}{\text{VInt}(z_t) \bowtie_I \text{VInt}(z_s)} \text{ VAL-INT} \qquad \frac{b_t = b_s}{\text{VBool}(b_t) \bowtie_I \text{VBool}(b_s)} \text{ VAL-BOOL} \\
\\
\frac{(l_t, l_s) \in I}{\text{VPtr}(l_t) \bowtie_I \text{VPtr}(l_s)} \text{ VAL-PTR} \qquad \frac{}{v_t \bowtie_I \text{VUndef}} \text{ VAL-UNDEF}
\end{array}$$

Figure 4: The value similarity predicate.

In RTL, registers store values that can be actively manipulated in computations. They are referenced by an identifier $r \in \text{reg}$. We do not restrict the number of registers. An RTL program can also interact with the heap. The heap is indexed by locations $l \in \text{loc}$, and the program can read, write, allocate, and free memory cells associated with locations.

3.2 Syntax and Program Structure

High-level imperative languages employ *structured* control-flow constructs, such as nested conditionals, while loops, and compound statements. In structured languages, execution paths follow the hierarchical source code structure of the program. In contrast, RTL is an *unstructured* language. Control flow in RTL is represented as an explicit directed graph whose nodes $pc \in \text{node}$ represent program points. Each instruction explicitly specifies the program points of its successors, eliminating the need for loop or block constructs. The instruction set and operator grammar of RTL are defined in fig. 5. An RTL function f is a record containing:

$$f \in \text{rtl_function} ::= \left\{ \begin{array}{ll} \text{fn_name} \in \text{ident} & \text{function identifier} \\ \text{fn_regs} \in \text{list reg} & \text{parameter list} \\ \text{fn_entrypoint} \in \text{node} & \text{function entry point} \\ \text{fn_code} \in \text{node} \rightarrow \text{instr} & \text{control-flow graph} \end{array} \right\}$$

We enforce the invariant that input parameter registers contain no duplicates. The code map fn_code is a finite map associating program points with instructions. In the following, we write $f[pc] = \langle i \rangle$ to assert that the code map of f contains instruction i at program point pc . An RTL program P consists of a list of functions with a distinguished entry function. We require that function names in the program are unique, and that the entry function is present.

$i \in \text{instr}$	$::=$	$\text{nop} \rightarrow pc$	unconditional jump
		$dst := \text{op } \vec{r} \rightarrow pc$	pure arithmetic or logical operation
		$dst := !src \rightarrow pc$	heap memory load
		$!addr := src \rightarrow pc$	heap memory store
		$dst := \text{alloc}() \rightarrow pc$	heap cell allocation
		$\text{free } addr \rightarrow pc$	heap cell deallocation
		$\text{if } cond \text{ then } pc_{\text{true}} \text{ else } pc_{\text{false}}$	conditional branching
		$dst := \text{call } fn(\vec{r}) \rightarrow pc$	function invocation
		$\text{ret } src$	function return
op	$::=$	$\text{Add} \mid \text{Sub} \mid \text{Mul} \mid \text{Div}$	integer arithmetic
		$\text{Incr} \mid \text{Decr}$	increment and decrement
		$\text{ImmInt}(z) \mid \text{ImmBool}(b)$	constant immediate values
		$\text{Move} \mid \text{EqZ}$	register copy and zero test

where $dst, src, addr, cond \in \text{reg}$, $\vec{r} \in \text{list reg}$, $fn \in \text{ident}$, and $pc, pc_{\text{true}}, pc_{\text{false}} \in \text{node}$.

Figure 5: The syntax of RTL instructions and operators.

3.3 Small-Step Operational Semantics

The execution of an RTL program is formalized as a small-step operational transition system. The configurations are tuples (σ, s, m) , where σ is a call stack, $m \in \text{memory}$ is the global heap, and $s \in \text{pcstate}$ is a control state which can take three distinct forms:

360 • State (f, pc, ρ) is the main state. It refers to an execution occurring inside function f at program point pc ,
 361 with local register bank ρ .
 362 • CallState $(f, \vec{v})$ represents the beginning of the invocation of function f with argument values $\vec{v}$.
 363 • ReturnState (v) represents the completion of a function execution with return value v .
 364 The call stack σ is a list of stack frames. Each stack frame Stackframe (dst, f, pc, ρ) stores the caller context: the
 365 destination register dst designated to receive the return value, the caller function f , the return program point
 366 pc , and the caller register bank ρ . Structuring transitions through explicit CallState $(f, \vec{v})$ and ReturnState (v)
 367 configurations cleanly decouples caller and callee environments. In CompCert [16], this abstraction is also used
 368 to integrate external function calls that jump directly from a call state to a return state.
 369 A register bank and the memory heap are formalized as shown in fig. 6. By explicitly recording Freed cells rather
 370 than removing locations from the domain of m , our memory model distinguishes dangling pointer dereferences
 371 and double frees from accesses to unallocated memory.

$\rho \in \text{regbank}$	$::=$	$\text{reg} \rightarrow \text{val}$	register bank
$m \in \text{memory}$	$::=$	$\text{loc} \rightarrow \text{cell}$	memory heap
$c \in \text{cell}$	$::=$	$\text{Allocated}(v) \mid \text{Freed}$	heap memory cell

Notation	Operation Semantics
$\rho[r]$	Read register r ; yields VUndef if $r \notin \text{dom}(\rho)$
$\rho[\vec{r}]$	Pointwise extension to register lists, returning $\vec{v}$
$\rho[r \mapsto v]$	Register assignment updating r to value v
$m[l]$	Read cell at location l
$m[l \mapsto \text{Allocated}(v)]$	Update allocated cell at l to value v
$m \uplus \{l \mapsto \text{Allocated}(v)\}$	Allocate fresh cell $l \notin \text{dom}(m)$ initialized with value v
$m[l \mapsto \text{Freed}]$	Deallocate active cell at l , setting its status to Freed

Figure 6: Formalization of register banks and memory heap operations.

$$\begin{array}{c}
 \frac{f[pc] = \langle dst := op \ \vec{r} \rightarrow pc' \rangle \quad \rho[\vec{r}] = \vec{v} \quad \text{eval_op}(op, \vec{v}) = \text{Some}(v)}{P \vdash (\sigma, \text{State}(f, pc, \rho), m) \rightarrow (\sigma, \text{State}(f, pc', \rho[dst \mapsto v]), m)} \text{ STEP-OP} \\
 \\
 \frac{f[pc] = \langle dst := \text{alloc}() \rightarrow pc' \rangle \quad l \notin \text{dom}(m) \quad v \in \text{val}}{P \vdash (\sigma, \text{State}(f, pc, \rho), m) \rightarrow (\sigma, \text{State}(f, pc', \rho[dst \mapsto \text{VPtr}(l)]), m \uplus \{l \mapsto \text{Allocated}(v)\})} \text{ STEP-ALLOC} \\
 \\
 \frac{f[pc] = \langle \text{free } addr \rightarrow pc' \rangle \quad \rho[addr] = \text{VPtr}(l) \quad m[l] = \text{Allocated}(_) }{P \vdash (\sigma, \text{State}(f, pc, \rho), m) \rightarrow (\sigma, \text{State}(f, pc', \rho), m[l \mapsto \text{Freed}])} \text{ STEP-FREE} \\
 \\
 \frac{f[pc] = \langle dst := \text{call } fn(\vec{r}) \rightarrow pc' \rangle \quad \text{find_fun}(P, fn) = \text{Some}(f_{\text{callee}}) \quad \rho[\vec{r}] = \vec{v}}{P \vdash (\sigma, \text{State}(f, pc, \rho), m) \rightarrow (\text{Stackframe}(dst, f, pc', \rho) :: \sigma, \text{CallState}(f_{\text{callee}}, \vec{v}), m)} \text{ STEP-CALL} \\
 \\
 \frac{|\vec{v}| = |\text{fn_regs}(f)| \quad \rho = \text{init_regs}(\text{fn_regs}(f), \vec{v})}{P \vdash (\sigma, \text{CallState}(f, \vec{v}), m) \rightarrow (\sigma, \text{State}(f, \text{fn_entrypoint}(f), \rho), m)} \text{ STEP-ENTRY} \\
 \\
 \frac{f[pc] = \langle \text{ret } src \rangle \quad \rho[src] = v}{P \vdash (\sigma, \text{State}(f, pc, \rho), m) \rightarrow (\sigma, \text{ReturnState}(v), m)} \text{ STEP-RET} \\
 \\
 \frac{}{P \vdash (\text{Stackframe}(dst, f, pc', \rho) :: \sigma, \text{ReturnState}(v), m) \rightarrow (\sigma, \text{State}(f, pc', \rho[dst \mapsto v]), m)} \text{ STEP-RETURN}
 \end{array}$$

Figure 7: Selected rules of the small-step operational semantics of RTL.

372 The step relation $P \vdash s \rightarrow s'$ is defined by inference rules, of which are presented in fig. 7. Execution of a
 373 function progresses through three main phases ($\text{CallState} \rightarrow \text{State} \rightarrow \text{ReturnState}$):

- 374 1. In the beginning, the state is a call state $\text{CallState}(f, \vec{v})$. We use the rule STEP-ENTRY to step into the body
 375 of the function at its entry point. This initializes the register bank with the values given for the input
 376 registers of f . The resulting state is an active state $\text{State}(f, pc_0, \rho)$.
- 377 2. Then we use the instruction rules to execute the body of the function. These rules can only apply if they
 378 correspond to the next instruction to be executed, referenced by pc in f . We note that evaluation of some
 379 instructions is partial. For instance, in the STEP-OP rule, if $\text{eval_op}(op, \vec{v})$ returns None , the execution is
 380 stuck.
- 381 3. When the function reaches a return instruction, the rule STEP-RET is applied. It evaluates its return register
 382 and enters a return state $\text{ReturnState}(v)$.
- 383 4. If the call stack is not empty, the caller resumes execution via STEP-RETURN , popping the top stack frame,
 384 writing the return value into register dst , and branching to the next pc .
- 385 5. When a function call occurs, the rule STEP-CALL is used. The caller evaluates the argument registers, pushes
 386 a fresh stack frame recording its continuation point pc' and register bank ρ , and transitions to a call state
 387 $\text{CallState}(f_{\text{callee}}, \vec{v})$.

388 An execution state is final when the call stack is empty and the program is in a return state. The final value is the
 389 one stored in the control state.

390 3.4 Stack Modularity and Execution Invariants

391 Reasoning directly about executions with arbitrary call stacks introduces unnecessary proof complexity. Without
 392 stack modularity, one would need to maintain an explicit invariant over caller stack frames and prove that it is
 393 preserved across every step. To achieve modularity, we establish lemmas showing that function executions can
 394 be analyzed in isolation with an empty call stack $\sigma = []$, and then embedded into arbitrary execution contexts.

395 When verifying a function call, the callee is verified starting from an empty call stack $([], \text{CallState}(f, \vec{v}), m)$.
 396 Once the callee reduces to $([], \text{ReturnState}(v), m')$, the derivation is lifted back into the caller's stack context,
 397 reconnecting the callee's execution with the caller's continuation.

398 4 Relational Separation Logic

399 Having defined our free simulation framework and instantiated it with the Register Transfer Language, we require
 400 a program logic to specify and verify relational properties of program executions. Reasoning about memory
 401 transformations using global heap predicates imposes a heavy verification burden, as every intermediate step
 402 must maintain global invariants. To enable local reasoning over pairs of memory heaps, we develop a relational
 403 separation logic.

404 4.1 Separation Logic

405 Separation logic [3] establishes a foundation for modular reasoning about programs that manipulate memory. In
 406 standard Hoare logic, assertions describe properties of the entire heap. Establishing that an operation modifies
 407 only a restricted portion of memory requires explicitly proving that all untouched heap regions remain invariant,
 408 an issue known as the frame problem. Separation logic solves this challenge by introducing the separating
 409 conjunction $P * Q$, which asserts that the memory can be split into two disjoint sub-heaps satisfying P and Q ,
 410 respectively.

411 In verified compilation, we reason relationally about two program executions: the target program and the source
 412 program. Because each execution operates over its own memory heap, we formulate a relational separation
 413 logic [28] whose assertions describe pairs of memory states (m_t, m_s) .

414 In modern interactive proof assistants, the Iris framework [10, 14] serves as the standard platform for separation
 415 logic in Rocq. Iris provides expressive mechanisms for higher-order concurrent reasoning, including step-indexed
 416 models, higher-order ghost state, and user-defined resource algebras. For our setting, these advanced features are

not needed. Instead, we formalize our relational separation logic directly as a shallow embedding in Rocq. We define the type of relational propositions, denoted rProp , as a binary relation over memories:

$$\text{rProp} := \text{memory} \rightarrow \text{memory} \rightarrow \text{Prop}$$

By convention, the first argument denotes the target memory m_t , and the second argument denotes the source memory m_s . An assertion $P \in \text{rProp}$ represents a predicate that expresses ownership over the memory footprints of both programs, specifying the contents of sub-heaps of the target and source.

Connective	Semantics	Description
$\ulcorner \psi \urcorner$	$\lambda m_t m_s. m_t = \emptyset \wedge m_s = \emptyset \wedge \psi$	Pure assertion
emp	$\lambda m_t m_s. m_t = \emptyset \wedge m_s = \emptyset$	Empty memory heaps
$P \wedge Q$	$\lambda m_t m_s. P(m_t, m_s) \wedge Q(m_t, m_s)$	Conjunction
$P \vee Q$	$\lambda m_t m_s. P(m_t, m_s) \vee Q(m_t, m_s)$	Disjunction
$P \rightarrow Q$	$\lambda m_t m_s. P(m_t, m_s) \implies Q(m_t, m_s)$	Implication
$\forall x, P(x)$	$\lambda m_t m_s. \forall x, P(x)(m_t, m_s)$	Universal quantification
$\exists x, P(x)$	$\lambda m_t m_s. \exists x, P(x)(m_t, m_s)$	Existential quantification
$P * Q$	$\lambda m_t m_s. \exists m_t^P m_t^Q m_s^P m_s^Q, m_t = m_t^P \uplus m_t^Q \wedge m_s = m_s^P \uplus m_s^Q$ $\wedge P(m_t^P, m_s^P) \wedge Q(m_t^Q, m_s^Q)$	Separating conjunction
$P \multimap Q$	$\lambda m_t m_s. \forall m_t', m_s', P(m_t', m_s') \implies Q(m_t \uplus m_t', m_s \uplus m_s')$	Magic wand
$\Box P$	$\lambda m_t m_s. P(\emptyset, \emptyset)$	Persistent modality
$P \vdash Q$	$\forall m_t m_s, P(m_t, m_s) \implies Q(m_t, m_s)$	Logical entailment
$P \dashv\vdash Q$	$(P \vdash Q) \wedge (Q \vdash P)$	Logical equivalence

Figure 8: Base connectives of our relational separation logic.

The semantic definitions of our base logical connectives are summarized in fig. 8. The entailment $P \vdash Q$ asserts that whenever P holds on a memory pair, Q holds as well. Logical equivalence $P \dashv\vdash Q$ holds when both directions of entailment hold. The connectives satisfy the standard algebraic properties of separation logic:

- The separating conjunction $P * Q$ splits both the target heap m_t and the source heap m_s into disjoint sub-heaps³, assigning one part to P and the other part to Q . Because the sub-heaps must be strictly disjoint, this operator prevents duplicate resource ownership.
- The persistent modality $\Box P$ asserts that P holds on empty heaps. Because a persistent assertion requires no memory resources, it represents a duplicable fact: $\Box P \vdash \Box P * \Box P$.

Our memory logic is *linear* rather than affine. In an affine logic, heap chunks can be silently discarded via weakening: $P * Q \vdash P$. In our linear formulation, proving $P \vdash Q$ guarantees that every memory cell described by P is accounted for in Q . This strict resource discipline forbid any memory leak.

To preserve the interactive proof engineering benefits of the Iris Proof Mode (MoSeL) [13], we prove that our connectives satisfy the required typeclass hierarchy. This allows our formalization to use standard Iris tactics while executing directly on our shallow relational model.

4.2 Relational Assertions & Memory Predicates

Building on our memory model (fig. 6), we introduce primitive memory assertions for target and source states. We define relational points-to and freed assertions:

$$\begin{aligned}
l \rightarrow_t v &:= \lambda m_t m_s. m_t = \{l \mapsto \text{Allocated}(v)\} \wedge m_s = \emptyset \\
l \rightarrow_s v &:= \lambda m_t m_s. m_t = \emptyset \wedge m_s = \{l \mapsto \text{Allocated}(v)\} \\
\text{free}_t(l) &:= \lambda m_t m_s. m_t = \{l \mapsto \text{Freed}\} \wedge m_s = \emptyset \\
\text{free}_s(l) &:= \lambda m_t m_s. m_t = \emptyset \wedge m_s = \{l \mapsto \text{Freed}\}
\end{aligned}$$

The predicate $l \rightarrow_t v$ asserts that location l in the target heap contains value v , while requiring the source heap to be empty. Symmetrically, $l \rightarrow_s v$ asserts ownership of an allocated cell in the source heap. The predicates $\text{free}_t(l)$ and $\text{free}_s(l)$ assert ownership of deallocated memory cells in the target and source heaps, respectively.

³Here $\uplus$ denotes the disjoint union of memory heaps, which is defined only when its operand domains are disjoint.

Memory allocations in the target and source programs are not guaranteed to produce identical physical addresses. To account for address variations, we maintain an address mapping $I \subseteq \text{loc} \times \text{loc}$, where each pair $(l_t, l_s) \in I$ relates a target location l_t to a source location l_s . We enforce that I is a bijective mapping: target locations are uniquely related to source locations. This mapping I parameterizes the value similarity relation $v_t \bowtie_I v_s$ (fig. 4). As the bijection grows monotonically with fresh memory allocations, the value relation is preserved:

Lemma 3 (Monotonicity of $\bowtie_I$). If $I \subseteq I'$, then $v_t \bowtie_I v_s \implies v_t \bowtie_{I'} v_s$.

To relate entire target and source heaps across program execution, we adapt the memory injection predicate $\text{meminj}_E(I)$ defined by Gähler et al. [6]. When the set E is empty, this predicate owns all memory cells registered in I . It asserts that every active pair of locations $(l_t, l_s) \in I$ holds relationally similar values, and that deallocated cells are consistently marked as freed in both heaps.

When verifying an instruction that reads, writes, or deallocates a specific pair of locations $(l_t, l_s) \in I$, we need to access the underlying points-to or freed assertion. However, removing (l_t, l_s) from the mapping I would shrink I , breaking its monotonicity and invalidating the value similarity of existing pointers. To overcome this difficulty, we introduce the *exploited set* $E \subseteq I$. The set E tracks the address pairs whose individual heap assertions have been temporarily extracted from the global injection for local reasoning.

The interaction between the memory injection predicate, the bijection I , and the exploited set E is primarily governed by three lemmas:

- **Exploitation:** When $(l_t, l_s) \in I$ is unexploited, the assertion $\text{meminj}_E(I)$ can be opened to extract ownership of (l_t, l_s) while updating the exploited set to $E \cup \{(l_t, l_s)\}$.
- **Reintegration:** Once a memory operation completes, the updated points-to assertion $l_t \rightarrow_t v_t * l_s \rightarrow_s v_s$ (with $v_t \bowtie_I v_s$) or freed assertion $\text{free}_t(l_t) * \text{free}_s(l_s)$ can be released back into the injection, decreasing the exploited set to $E \setminus \{(l_t, l_s)\}$.
- **Extension:** When allocating fresh memory cells, the fresh pair of addresses (l_t, l_s) can be added to the memory injection provided their values are related by $\bowtie_I$.

4.3 Relational Hoare Quadruples

By combining the free simulation with relational separation logic, we define relational Hoare quadruples. In fixed target and source programs P_t and P_s , a relational Hoare quadruple relates a target function f_t to a source function f_s . The formal definition is given by:

$$\{P\} f_t \lesssim f_s \{Q\} := \Box \left(\forall \vec{v}_t \vec{v}_s, P(\vec{v}_t, \vec{v}_s) * (\Box, \text{CallState}(f_t, \vec{v}_t)) \mathrel{\mathbf{0} \lesssim \mathbf{0}} (\Box, \text{CallState}(f_s, \vec{v}_s)) \{Q\} \right)$$

where P is a precondition relating the target and source argument vectors, and Q is a postcondition referring to the returned values. The simulation is here lifted to an assertion in rProp :

$$t \mathrel{j \lesssim_i} s \{ \Phi \} := \lambda m_t m_s. (t, m_t) \mathrel{j \lesssim_i} (s, m_s) \{ \Phi \}$$

The meaning of this quadruple is given by the free simulation: whenever the initial function arguments and heap states satisfy $P(\vec{v}_t, \vec{v}_s)$, if the source execution terminates without encountering undefined behavior, then the target execution also terminates with return values and memories satisfying $Q(v_t, v_s)$. If the source execution diverges, the target execution diverges as well. If the source execution reaches a stuck state, the simulation relation grants the compiler complete freedom.

The definition of the Hoare quadruple is guarded by a persistent modality $\Box$ to guarantee that the specification holds independently of heap ownership. As a consequence, a verified quadruple constitutes a duplicable resource that can be reused across multiple call sites throughout the program.

5 RTL Proof Rules

Having established a simulation relation, the semantics of the Register Transfer Language, and a separation logic over pairs of memories, we combine these components into a relational proof system. To facilitate the verification of code transformations, we establish a structured collection of relational proof rules operating directly on the simulation judgment $t \mathrel{j \lesssim_i} s \{ \Phi \}$. These rules are all proved sound with respect to the simulation relation and the RTL semantics.

5.1 Asynchronous Rules

Our simulation relation decouples the execution of the target program from that of the source program. This separation yields asynchronous proof rules that advance one program while keeping the configuration of the other fixed.

Target proof rules express the conditions under which the target program can execute an instruction. The rules governing target execution of allocations and operations are displayed in fig. 9. The source stepping rules mirror the target rules, operating on the source program counter, source registers, and source memory assertions.

$$\begin{array}{c}
 \text{eval_op}(\text{op}, \vec{v}) = \text{Some}(v) \quad \rho_t[\vec{r}] = \vec{v} \\
 \frac{f_t[pc_t] = \langle dst := \text{op } \vec{r} \rightarrow pc \rangle \quad (\sigma_t, \text{State}(f_t, pc, \rho_t[dst \mapsto v])) \text{ } j' \lesssim_i s_s \{ \Phi \}}{(\sigma_t, \text{State}(f_t, pc_t, \rho_t)) \text{ } j \lesssim_i s_s \{ \Phi \}} \quad \text{TARGET-OP} \\
 \\
 \frac{\forall l, l \rightarrow_t v \multimap (\sigma_t, \text{State}(f_t, pc, \rho_t[dst \mapsto \text{VPtr}(l)])) \text{ } j' \lesssim_i s_s \{ \Phi \}}{(\sigma_t, \text{State}(f_t, pc_t, \rho_t)) \text{ } j \lesssim_i s_s \{ \Phi \}} \quad \text{TARGET-ALLOC}
 \end{array}$$

Figure 9: Asynchronous rules for the allocation and operations instruction in target program.

A key semantic difference between target and source execution appears in the allocation rule TARGET-ALLOC versus SOURCE-ALLOC. In our operational semantics, allocating a fresh memory cell non-deterministically initializes its content with an arbitrary value. Because the target is demonic, the continuation in TARGET-ALLOC must hold universally for *all* possible fresh locations l and all initial values v . In contrast, because the source is angelic, when applying SOURCE-ALLOC, the value of the allocated memory cell may be chosen existentially:

$$\begin{array}{c}
 v \in \text{val} \\
 \frac{\forall l, l \rightarrow_s v \multimap s_t \text{ } j \lesssim_{i'} (\sigma_s, \text{State}(f_s, pc, \rho_s[dst \mapsto \text{VPtr}(l)])) \{ \Phi \}}{s_t \text{ } j \lesssim_i (\sigma_s, \text{State}(f_s, pc_s, \rho_s)) \{ \Phi \}} \quad \text{SOURCE-ALLOC}
 \end{array}$$

5.2 Undefined Behavior Exploitation

By the FSOURCESTUCK rule, our free simulation is skewed. When the source program reaches a configuration from which no further transition is possible without being in a final state, the simulation property $t \text{ } j \lesssim_i s \{ \Phi \}$ holds directly. This property provides a powerful proof technique: whenever the source program is about to perform an operation, we can safely assume that the operation succeeds. If the operation were to fail, the source would be stuck, and the proof obligation would be instantly discharged. For instance, the exploit rule SOURCE-OP-EXPLOIT allows the proof to assume that the operation succeeds and produces a value v :

$$\begin{array}{c}
 \rho_s[\vec{r}] = \vec{v} \\
 \frac{\forall v, \ulcorner \text{eval_op}(\text{op}, \vec{v}) = \text{Some}(v) \urcorner \multimap s_t \text{ } j \lesssim_{1+i} (\sigma_s, \text{State}(f_s, pc, \rho_s[dst \mapsto v])) \{ \Phi \}}{s_t \text{ } j \lesssim_i (\sigma_s, \text{State}(f_s, pc_s, \rho_s)) \{ \Phi \}} \quad \text{SOURCE-OP-EXPLOIT}
 \end{array}$$

5.3 Exploiting Memory Operations

In separation logic, memory resources cannot be obtained without explicit allocation. However, when programs manipulate values related through a memory injection $\text{meminj}_E(I)$, heap assertions can be extracted directly from the global injection invariant.

Suppose the source program is going to execute a load on an address addr_s that is related to a target address through $\text{addr}_t \bowtie_I \text{addr}_s$. Two deductions follow:

1. **Pointer validity:** If addr_s is not a valid pointer $\text{VPtr}(l_s)$, the source memory access fails and becomes stuck. Therefore, from the relation $\text{addr}_t \bowtie_I \text{addr}_s$, we deduce that $\text{addr}_s = \text{VPtr}(l_s)$ and $\text{addr}_t = \text{VPtr}(l_t)$ with $(l_t, l_s) \in I$.

2. **Active status:** Opening the memory injection $\text{meminj}_\emptyset(I)$ via the exploitation lemma reveals that the cell pair (l_t, l_s) is either actively allocated or already freed. However, if the location were freed, attempting a load in the source would produce undefined behavior. Thus, the freed case is instantly discharged, and the cell is guaranteed to be currently allocated.

Because we exploited the location pair $(l_t, l_s) \in I$, the set E in the returned memory injection is augmented. This reasoning is formalized in the rule SOURCE-LOAD-EXPLOIT shown in fig. 10. Similar exploitation principles apply to the remaining memory operations.

$$\begin{array}{c}
 f_s[pc_s] = \langle dst := !src \rightarrow pc \rangle \\
 \rho_s[src] = addr_s \quad addr_t \bowtie_I addr_s \quad \text{meminj}_\emptyset(I) \\
 \left(\begin{array}{l}
 \forall l_t l_s v_t v_s, \quad addr_s = \text{VPtr}(l_s) \wedge addr_t = \text{VPtr}(l_t) \implies \\
 l_t \rightarrow_t v_t * l_s \rightarrow_s v_s * \ulcorner v_t \bowtie_I v_s \urcorner * \text{meminj}_{\{(l_t, l_s)\}}(I) -* \\
 s_t \lesssim_{i'} (\sigma_s, \text{State}(f_s, pc, \rho_s[dst \mapsto v_s])) \{\Phi\}
 \end{array} \right) \\
 \hline
 s_t \lesssim_i (\sigma_s, \text{State}(f_s, pc_s, \rho_s)) \{\Phi\}
 \end{array} \quad \text{SOURCE-LOAD-EXPLOIT}$$

Figure 10: Memory exploitation rule for source load instructions.

5.4 Synchronous Rules

While asynchronous and exploit rules provide flexibility for transformations that reorder or eliminate instructions, many compiler passes preserve the overall control flow of the program. For such passes, stepping target and source programs independently introduces redundant bookkeeping. By composing source exploit rules with target stepping rules, we obtain synchronous rules that execute matching instructions in lockstep.

This is notably the case when reasoning about synchronous function calls. To ensure modularity, we avoid explicitly stepping through the execution of callee bodies. Instead, we rely on the Hoare quadruples introduced in section 4.3. The synchronous call rule BOTH-CALL enables replacing a pair of matching function calls by their relational specification. Its structure highlights the core benefits of our relational separation logic.

Suppose both programs are about to execute a call:

- the target calls function f'_t with argument values $\vec{v}_t$, where $f_t[pc_t] = \langle dst_t := \text{call } f'_t(\vec{a}_t) \rightarrow pc'_t \rangle$ and $\rho_t[\vec{a}_t] = \vec{v}_t$;
- the source calls function f'_s with argument values $\vec{v}_s$, where $f_s[pc_s] = \langle dst_s := \text{call } f'_s(\vec{a}_s) \rightarrow pc'_s \rangle$ and $\rho_s[\vec{a}_s] = \vec{v}_s$;
- a relational specification $\{P\} f'_t \lesssim f'_s \{Q\}$ has already been established for the called functions.

Under these conditions, the BOTH-CALL rule proceeds as follows:

1. The caller must prove that current heap resources satisfy the relational precondition $P(\vec{v}_t, \vec{v}_s)$.
2. Instead of stepping through the internal instructions of f'_t and f'_s , the rule bypasses the callee bodies entirely.
3. Execution resumes at the return points pc'_t and pc'_s with the destination registers updated with return values, satisfying the postcondition $Q(v_t, v_s)$.

Because Hoare quadruples are enclosed in the persistent modality, they represent duplicable specifications that can be verified once per function, and reused across arbitrarily many call sites. This provides horizontal modularity: functions in a program can be compiled and verified independently, and their correctness guarantees compose cleanly through BOTH-CALL.

6 Case Studies

To evaluate the relational separation logic developed in this work, we present three case studies: a local transformation exploiting undefined behavior, a loop transformation hoisting an invariant load, and a control-flow optimization eliminating redundant jump chains.

6.1 Undefined Behavior Exploitation

We begin with a local program transformation that eliminates a redundant zero-check after an arithmetic division. Consider the source and target code fragments in fig. 11.

Source Program		Target Program
1 : $r_1 := \text{Div } [r_n, r_d] \rightarrow 2$	$\implies$	1 : $r_1 := \text{Div } [r_n, r_d] \rightarrow 2$
2 : $r_b := \text{EqZ } [r_d] \rightarrow 3$		2 : $r_b := \text{ImmBool}(\text{false}) \rightarrow 3$
3 : ...		3 : ...

Figure 11: Exploiting division undefined behavior to simplify a zero test.

In the source program, instruction 1 divides the content of r_n by r_d , storing the quotient in r_1 . Instruction 2 tests whether the divisor r_d is zero via EqZ , storing the resulting boolean in r_b . In the target program, instruction 2 replaces this test with an immediate assignment of false.

The soundness of this transformation relies on the asymmetric nature of our simulation relation. When reasoning about instruction 1, we apply the rule SOURCE-OP-EXPLOIT . Because stuck source executions satisfy the simulation relation by FSOURCESTUCK , the rule provides the hypothesis that the operation succeeds: $\text{eval_op}(\text{Div}, [\rho_s(r_n), \rho_s(r_d)]) = \text{Some}(v)$. Under our operational semantics, division succeeds only when the divisor $\rho_s(r_d) = \text{VInt}(z_d)$ satisfies $z_d \neq 0$. Assuming that target and source register banks agree initially, the target division executes identically, advancing both programs to $pc = 2$.

At program point 2, the source program tests register r_d for equality to zero. Because $\rho_s[r_d] = \text{VInt}(z_d)$ with $z_d \neq 0$, this evaluation deterministically yields false. In the target program, the immediate load executes via TARGET-OP and writes false into r_b without inspecting r_d . Both programs reach $pc = 3$ in matching register and memory states, establishing the soundness of this transformation.

6.2 Load Hoisting

We next examine a transformation hoisting a load out of a loop. Figure 12 displays two versions of a factorial function $\text{fact}(n, \text{addr})$. The function initializes the memory cell at addr to 1, and computes the factorial of n .

Source Function (fact_bad)	Target Function (fact_good)
0 : $r_{res} := \text{ImmInt}(1) \rightarrow 1$	0 : $r_{res} := \text{ImmInt}(1) \rightarrow 1$
1 : $!r_{addr} := r_{res} \rightarrow 2$	1 : $!r_{addr} := r_{res} \rightarrow 2$
2 : $r_{cond} := \text{EqZ } [r_n] \rightarrow 3$	2 : $r_{one} := \text{ImmInt}(1) \rightarrow 3$
3 : if r_{cond} then 7 else 4	3 : $r_{cond} := \text{EqZ } [r_n] \rightarrow 4$
4 : $r_{res} := \text{Mul } [r_{res}, r_n] \rightarrow 5$	4 : if r_{cond} then 7 else 5
5 : $r_{one} := !r_{addr} \rightarrow 6$	5 : $r_{res} := \text{Mul } [r_{res}, r_n] \rightarrow 6$
6 : $r_n := \text{Sub } [r_n, r_{one}] \rightarrow 2$	6 : $r_n := \text{Sub } [r_n, r_{one}] \rightarrow 3$
7 : ret r_{res}	7 : ret r_{res}

Figure 12: Source and target factorial functions illustrating load hoisting.

In the unoptimized function fact_bad , the loop body performs a redundant heap load at $pc = 5$ on each iteration, reading the constant 1 stored at r_{addr} to decrement r_n . The optimized function fact_good hoists the load into the prelude. It further replaces the memory access with an immediate assignment of 1 into register r_{one} .

To prove the soundness of this transformation, we prove the following relational Hoare quadruple for all heap bijections I :

$$\{\text{same_args}_I\} \text{fact_good} \lesssim \text{fact_bad} \{\text{samer}_I\} \quad (5)$$

574 where same_args_I and samer_I relate input arguments and return values under an extendable heap bijection:

$$\begin{aligned}\text{same_args}_I(\vec{v}_t, \vec{v}_s) &:= \lceil \text{Forall2}(\bowtie_I, \vec{v}_t, \vec{v}_s) \rceil * \text{meminj}_{\emptyset}(I) \\ \text{samer}_I(v_t, v_s) &:= \exists I', \lceil I \subseteq I' \rceil * \lceil v_t \bowtie_{I'} v_s \rceil * \text{meminj}_{\emptyset}(I')\end{aligned}$$

575 The precondition $\text{same_args}_I(\vec{v}_t, \vec{v}_s)$ asserts that input arguments are pairwise related under I , alongside an
576 unexploited memory injection $\text{meminj}_{\emptyset}(I)$. The postcondition $\text{samer}_I(v_t, v_s)$ guarantees that return values
577 remain related under a monotonically extended bijection $I' \supseteq I$, with the memory injection $\text{meminj}_{\emptyset}(I')$ fully
578 restored. Allowing the bijection to grow monotonically accounts for any fresh memory allocations performed
579 during execution.

580 The proof of eq. (5) begins by stepping through the prelude. Both programs initialize r_{res} to 1. Applying SOURCE-
581 STORE-EXPLOIT extracts (l_t, l_s) from $\text{meminj}_{\emptyset}(I)$, yielding $l_s \rightarrow_s \text{VInt}(1)$ and advancing the source to $pc_s = 2$.
582 Applying TARGET-STORE then executes the target store, producing $l_t \rightarrow_t \text{VInt}(1)$. Finally, the target initializes
583 r_{one} via TARGET-OP and reaches $pc_t = 3$, establishing the following loop invariant:

- 584 • The target program is executing `fact_good` at program counter $pc_t = 3$ with register bank ρ_t .
- 585 • The source program is executing `fact_bad` at program counter $pc_s = 2$ with register bank ρ_s .
- 586 • The register banks agree on the counter and accumulator: $\rho_t(r_n) \bowtie_I \rho_s(r_n)$ and $\rho_t(r_{res}) \bowtie_I \rho_s(r_{res})$.
- 587 • The registers r_{addr} hold related pointer values: $\rho_t(r_{addr}) = \text{VPtr}(l_t)$ and $\rho_s(r_{addr}) = \text{VPtr}(l_s)$ with
588 $(l_t, l_s) \in I$.
- 589 • The target register r_{one} holds the constant $\text{VInt}(1)$ in ρ_t .
- 590 • The memory state satisfies $l_t \rightarrow_t \text{VInt}(1) * l_s \rightarrow_s \text{VInt}(1) * \text{meminj}_{\{(l_t, l_s)\}}(I)$, isolating the hoisted cell
591 (l_t, l_s) while the remaining heap is governed by the injection.

592 The proof then proceeds by coinduction, evaluating the zero-test $\text{EqZ}(r_n) = b$ synchronously in both programs:

593 **Base case** ($b = \text{true}$): Both programs branch to the return instruction at $pc = 7$. The points-to assertions
594 $l_t \rightarrow_t \text{VInt}(1) * l_s \rightarrow_s \text{VInt}(1)$ are returned to the memory injection via the reintegration lemma, restoring
595 $\text{meminj}_{\emptyset}(I)$. Both programs execute `ret` r_{res} , satisfying the postcondition samer_I with $I' = I$.

596 **Step case** ($b = \text{false}$): Both programs execute the multiplication $r_{res} := r_{res} \times r_n$. At source $pc_s = 5$, the source
597 load $r_{one} := !r_{addr}$ executes via SOURCE-LOAD directly from $l_s \rightarrow_s \text{VInt}(1)$ without requiring memory
598 exploitation. Both programs then decrement $r_n := r_n - r_{one}$, preserving register agreement, and branch
599 back to the loop headers ($pc_t = 3$ and $pc_s = 2$). This re-establishes the invariant, closing the case by the
600 coinductive hypothesis.

601 6.3 Nop Tunneling

602 We now consider an optimization on arbitrary control-flow graphs: nop tunneling. Intermediate compiler passes
603 frequently introduce redundant jump instructions (`nop` $\rightarrow$ pc). A sequence of consecutive jumps forms a *nop*
604 *chain*:

$$pc_0 \rightarrow pc_1 \rightarrow pc_2 \rightarrow \dots \rightarrow pc_k \rightarrow pc_{\text{final}}$$

605 where each node pc_m ($0 \leq m < k$) contains `nop` $\rightarrow$ pc_{m+1} , and pc_{final} is the first non-nop instruction. Nop
606 tunneling short-circuits these chains by redirecting branch targets directly to pc_{final} , reducing instruction count
607 and eliminating superfluous jump overhead.

608 The transformation is defined in Rocq over an arbitrary RTL function f . To ensure termination, even in the
609 presence of cyclic jump chains (such as an infinite loop pc : `nop` $\rightarrow$ pc), the computation of the final successor of
610 an instruction at pc is parameterized by a recursion depth $d \in \mathbb{N}$:

$$\begin{aligned}\text{find_next_pc}(0, f, pc) &:= pc \\ \text{find_next_pc}(1 + d, f, pc) &:= \begin{cases} \text{find_next_pc}(d, f, pc') & \text{if } f[pc] = \langle \text{nop} \rightarrow pc' \rangle \\ pc & \text{otherwise} \end{cases}\end{aligned}$$

611 The optimization `tunnel_nop` (d, f) maps `find_next_pc` over every instruction in the control-flow graph of f ,
612 redirecting each successor label pc to `find_next_pc` (d, f, pc). The function entry point is updated in the same

manner. To establish the soundness of nop tunneling for an arbitrary function f , we prove the following relational Hoare quadruple for all depths d and heap bijections I :

$$\{\text{same_args}_I\} \text{ tunnel_nop}(d, f) \lesssim f \{\text{samer}_I\} \quad (6)$$

The proof relies on a coinductive simulation invariant. Given a fixed initial fuel $depth$, the invariant asserts that at any step of execution:

- The target program is executing the optimized function $\text{tunnel_nop}(depth, f)$ at program counter pc_t with register bank ρ_t .
- The source program is executing the original function f at program counter pc_s with register bank ρ_s .
- The target program counter is the tunneled successor of the source program counter with remaining fuel d : $pc_t = \text{find_next_pc}(d, f, pc_s)$.
- The register banks agree on all registers: $\forall r, \rho_t(r) \bowtie_I \rho_s(r)$.
- The memory heaps are fully synchronized under a monotone bijection I : $\text{meminj}_\emptyset(I)$.

The proof of eq. (6) proceeds by coinduction followed by an inner induction on the remaining fuel d :

Base case ($d = 0$): When fuel is exhausted, $\text{find_next_pc}(0, f, pc_s) = pc_s$, so $pc_t = pc_s$. The target and source are positioned at identical program points. By case analysis on the instruction at pc_s , both programs progress in lockstep for each instruction kind. The next target program counter is the redirected label $\text{find_next_pc}(depth, f, pc'_s)$, which re-establishes the invariant with full fuel $d = depth$. The coinductive hypothesis closes this case.

Inductive step ($d > 0$): We inspect the source instruction at pc_s :

- If pc_s maps to a jump $\text{nop} \rightarrow pc'_s$, then $\text{find_next_pc}(1 + d', f, pc_s) = \text{find_next_pc}(d', f, pc'_s)$. The source executes the jump via SOURCE-NOP, advancing to pc'_s . The target remains at pc_t . The inner induction hypothesis on d' concludes this case.
- If pc_s does not refer to a jump instruction, $\text{find_next_pc}(1 + d', f, pc_s) = pc_s$, so $pc_t = pc_s$. As in the base case, target and source progress in lockstep, resetting fuel to $depth$ for successors. The coinductive hypothesis closes this case.

When verifying function call instructions, the proof applies the synchronous BOTH-CALL rule. Because the coinductive invariant is parameterized over arbitrary RTL functions, it can be instantiated directly for the callee. Therefore, the relational Hoare quadruple required by BOTH-CALL is discharged without additional lemmas. This illustrates the modularity of our relational separation logic: function-level optimizations are verified in isolation and compose cleanly across whole-program call graphs.

7 Related Work

We compare our approach with existing verified compilers, simulation techniques, and relational separation logics.

7.1 Verified Compilers

Non-Determinism CompCert [16] is a verified C compiler that establishes correctness by proving forward simulation diagrams for each compilation phase. When the target language is deterministic, a forward simulation implies a backward simulation, which guarantees semantic preservation. However, concurrent extensions such as CompCertTSO [22] introduce non-determinism into target executions, requiring backward simulations to be established directly. CakeML [15, 25] avoids this complication by enforcing determinism throughout its source semantics and intermediate representations, disallowing concurrency and internal non-deterministic choice. In our framework, we directly establish a safe backward simulation. This enables our system to accommodate internal non-determinism in the target language. Nevertheless, our current RTL operational semantics features only limited non-determinism, specifically fresh memory allocation, with no support for concurrent primitives.

655 **Stuttering** CompCert handles stuttering by indexing simulation relations with an explicit well-founded order,
656 which must strictly decrease on stuttering steps. CakeML uses a fuel-indexed operational semantics. Because
657 evaluation in CakeML is deterministic, semantic preservation is established by case analysis on the existence of a
658 sufficient fuel value to reach termination. In our approach, we address stuttering coinductively through a free
659 simulation relation [4]. This formulation avoids constructing explicit well-founded metrics or managing fuel
660 bounds.

661 **Memory Models** CompCert uses a block-based memory model [17], in which program transformations that
662 alter memory layouts are justified through global memory injections. These injections map source memory
663 blocks to target blocks across the global heap. CakeML similarly tracks memory through a global mutable store,
664 lowering this store to a flat linear array in the backend. In our work, we employ separation logic to reason locally
665 about memory fragments and memory injection. Local heap assertions avoid reasoning globally about the entire
666 memory state. We hope that this formulation will ease future proofs of transformations impacting memory.

667 **Modularity** CompCert is structured as a whole-program compiler with global call-stack invariants. To support
668 verified separate compilation and modular linking, several extensions have been developed [24, 11]. Similarly,
669 while early CakeML proofs were whole-program, recent extensions support dynamic compilation primitives [23].
670 In our framework, relational Hoare quadruples provide horizontal compositionality, allowing verified code
671 components to be used independently. However, our current formalization operates within a single intermediate
672 language, leaving compilation passes between distinct languages to future work.

673 7.2 Relational Separation Logics

674 **Logical Foundations** Simuliris [6] is a relational separation logic framework implemented within Iris [10].
675 Recent work on Tail Modulo Cons (TMC) by Allain et al. [1] similarly embeds a relational separation logic in
676 Iris to verify the soundness of the TMC transformation for a functional language. Other relational frameworks
677 built on Iris include ReLoC [5] for contextual refinement in concurrent languages, and Melocoton [8] for verified
678 interoperability between OCaml and C.

679 While Iris provides substantial expressive power, its semantic model relies on complex constructions, such
680 as step-indexed resource algebras and modal update operators. In our framework, we do not build upon the
681 semantic model of Iris. Instead, we formulate our relational separation logic directly over concrete heap states
682 and coinductive simulation relations. Our logical definitions remain direct while maintaining compatibility with
683 MoSeL [13], the interactive separation logic proof mode.

684 **Program Representation and Control Flow** The nature of the underlying languages also distinguishes
685 our work from prior relational separation logics. Both Simuliris and TMC target high-level languages with
686 structured, syntax-directed control flow. In contrast, our framework supports a low-level register transfer
687 language where programs are represented as control-flow graphs. This requires relational proof rules capable of
688 handling unstructured control flow.

689 **Verified Transformations** The scope of verified program transformations also differs. Simuliris focuses on
690 fine-grained concurrent optimizations, such as code motion and instruction reordering under weak memory
691 models. TMC proves the correctness of a specialized source-to-source transformation, converting non-tail-
692 recursive functions into tail-recursive loops by mutating newly allocated constructor fields. In our work, we
693 evaluate our logic on backend compiler transformations over control-flow graphs, verifying optimizations that
694 eliminate redundant jump chains.

- [1] Clément Allain et al. “Tail Modulo Cons, OCaml, and Relational Separation Logic.” In: *Proc. ACM Program. Lang.* 9.POPL (2025), pp. 2337–2363. DOI: 10.1145/3704915.
- [2] José Bacelar Almeida et al. “Jasmin: High-Assurance and High-Speed Cryptography.” In: *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, CCS 2017, Dallas, TX, USA, October 30 - November 03, 2017*. Ed. by Bhavani Thuraisingham et al. ACM, 2017, pp. 1807–1823. DOI: 10.1145/3133956.3134078.
- [3] Arthur Charguéraud. “Separation logic for sequential programs (functional pearl).” In: *Proc. ACM Program. Lang.* 4.ICFP (2020), 116:1–116:34. DOI: 10.1145/3408998.
- [4] Minki Cho et al. “Stuttering for Free.” In: *Proc. ACM Program. Lang.* 7.OOPSLA2 (2023), pp. 1677–1704. DOI: 10.1145/3622857.
- [5] Dan Frumin, Robbert Krebbers, and Lars Birkedal. “ReLoC Reloaded: A Mechanized Relational Logic for Fine-Grained Concurrency and Logical Atomicity.” In: *Log. Methods Comput. Sci.* 17.3 (2021). DOI: 10.46298/LMCS-17(3:9)2021.
- [6] Lennard Gäher et al. “Simuliris: a separation logic framework for verifying concurrent program optimizations.” In: *Proc. ACM Program. Lang.* 6.POPL (2022), pp. 1–31. DOI: 10.1145/3498689. URL: <https://doi.org/10.1145/3498689>.
- [7] Eduardo Giménez. *A Tutorial on Recursive Types in Coq*. Research Report RT-0221. Projet COQ. INRIA, May 1998, p. 42. URL: <https://inria.hal.science/inria-00069950>.
- [8] Armaël Guéneau et al. “Melocoton: A Program Logic for Verified Interoperability Between OCaml and C.” In: *Proc. ACM Program. Lang.* 7.OOPSLA2 (2023), pp. 716–744. DOI: 10.1145/3622823.
- [9] Ralf Jung et al. *std++: An extended standard library for Rocq*. 2026. URL: <https://gitlab.mpi-sws.org/iris/stdpp/>.
- [10] Ralf Jung et al. “Iris from the ground up: A modular foundation for higher-order concurrent separation logic.” In: *J. Funct. Program.* 28 (2018), e20. DOI: 10.1017/S0956796818000151.
- [11] Jeehoon Kang et al. “Lightweight verification of separate compilation.” In: *Proceedings of the 43rd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2016, St. Petersburg, FL, USA, January 20 - 22, 2016*. Ed. by Rastislav Bodík and Rupak Majumdar. ACM, 2016, pp. 178–190. DOI: 10.1145/2837614.2837642.
- [12] Gerwin Klein and Tobias Nipkow. “A machine-checked model for a Java-like language, virtual machine, and compiler.” In: *ACM Trans. Program. Lang. Syst.* 28.4 (2006), pp. 619–695. DOI: 10.1145/1146809.1146811.
- [13] Robbert Krebbers et al. “MoSeL: a general, extensible modal framework for interactive proofs in separation logic.” In: *Proc. ACM Program. Lang.* 2.ICFP (2018), 77:1–77:30. DOI: 10.1145/3236772.
- [14] Robbert Krebbers et al. “The Essence of Higher-Order Concurrent Separation Logic.” In: *Programming Languages and Systems - 26th European Symposium on Programming, ESOP 2017, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2017, Uppsala, Sweden, April 22-29, 2017, Proceedings*. Ed. by Hongseok Yang. Lecture Notes in Computer Science. Springer, 2017, pp. 696–723. DOI: 10.1007/978-3-662-54434-1_26.
- [15] Ramana Kumar et al. “CakeML: a verified implementation of ML.” In: *The 41st Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL ’14, San Diego, CA, USA, January 20-21, 2014*. Ed. by Suresh Jagannathan and Peter Sewell. ACM, 2014, pp. 179–192. DOI: 10.1145/2535838.2535841.
- [16] Xavier Leroy. “Formal verification of a realistic compiler.” In: *Commun. ACM* 52.7 (2009), pp. 107–115. DOI: 10.1145/1538788.1538814.
- [17] Xavier Leroy and Sandrine Blazy. “Formal Verification of a C-like Memory Model and Its Uses for Verifying Program Transformations.” In: *J. Autom. Reason.* 41.1 (2008), pp. 1–31. DOI: 10.1007/S10817-008-9099-0.
- [18] Robin Milner. *Communication and concurrency*. PHI Series in computer science. Prentice Hall, 1989. ISBN: 978-0-13-115007-2.
- [19] Damien Pous. “Coinduction All the Way Up.” In: *Proceedings of the 31st Annual ACM/IEEE Symposium on Logic in Computer Science, LICS ’16, New York, NY, USA, July 5-8, 2016*. Ed. by Martin Grohe, Eric Koskinen, and Natarajan Shankar. ACM, 2016, pp. 307–316. DOI: 10.1145/2933575.2934564.
- [20] Damien Pous. *Coinduction: a library for doing proofs by (enhanced) coinduction*. Version 1.22. 2026. URL: <https://github.com/damien-pous/coinduction>.
- [21] Steven Schäfer and Gert Smolka. “Tower Induction and Up-to Techniques for CCS with Fixed Points.” In: *Relational and Algebraic Methods in Computer Science - 16th International Conference, RAMiCS 2017, Lyon,*

- France, May 15-18, 2017, *Proceedings*. Ed. by Peter Höfner, Damien Pous, and Georg Struth. Vol. 10226. Lecture Notes in Computer Science. 2017, pp. 274–289. doi: 10.1007/978-3-319-57418-9_17.
- [22] Jaroslav Sevcík et al. “Relaxed-memory concurrency and verified compilation.” In: *Proceedings of the 38th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2011, Austin, TX, USA, January 26-28, 2011*. Ed. by Thomas Ball and Mooly Sagiv. ACM, 2011, pp. 43–54. doi: 10.1145/1926385.1926393.
- [23] Thomas Sewell et al. “Cakes That Bake Cakes: Dynamic Computation in CakeML.” In: *Proc. ACM Program. Lang.* 7.PLDI (2023), pp. 1121–1144. doi: 10.1145/3591266.
- [24] Gordon Stewart et al. “Compositional CompCert.” In: *Proceedings of the 42nd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2015, Mumbai, India, January 15-17, 2015*. Ed. by Sriram K. Rajamani and David Walker. ACM, 2015, pp. 275–287. doi: 10.1145/2676726.2676985.
- [25] Yong Kiam Tan et al. “The verified CakeML compiler backend.” In: *J. Funct. Program.* 29 (2019), e2. doi: 10.1017/S0956796818000229.
- [26] The Rocq Development Team. *The Rocq Prover*. Formerly known as The Coq Proof Assistant. Inria. 2026. URL: <https://rocq-prover.org>.
- [27] Li-yao Xia et al. “Interaction trees: representing recursive and impure programs in Coq.” In: *Proc. ACM Program. Lang.* 4.POPL (2020), 51:1–51:32. doi: 10.1145/3371119.
- [28] Hongseok Yang. “Relational separation logic.” In: *Theor. Comput. Sci.* 375.1-3 (2007), pp. 308–334. doi: 10.1016/J.TCS.2006.12.036.

Contents

769	1 Verified Compilation Background	3
770	1.1 Small-Step Operational Semantics	3
771	1.2 Program Behaviors	3
772	1.3 Program Refinement	3
773	1.4 Simulation Relations	4
774	1.5 Coinductive Predicates	4
775	1.6 Simulation Schemes & Stuttering	5
776	2 The Free Simulation	6
777	2.1 Skewed Implicit Simulation	7
778	2.2 Step-Indexed Free Simulation	7
779	2.3 Properties	8
780	3 The Register Transfer Language (RTL)	9
781	3.1 Values, Registers, and Memory	9
782	3.2 Syntax and Program Structure	10
783	3.3 Small-Step Operational Semantics	10
784	3.4 Stack Modularity and Execution Invariants	12
785	4 Relational Separation Logic	12
786	4.1 Separation Logic	12
787	4.2 Relational Assertions & Memory Predicates	13
788	4.3 Relational Hoare Quadruples	14
789	5 RTL Proof Rules	14
790	5.1 Asynchronous Rules	15
791	5.2 Undefined Behavior Exploitation	15
792	5.3 Exploiting Memory Operations	15
793	5.4 Synchronous Rules	16
794	6 Case Studies	16
795	6.1 Undefined Behavior Exploitation	17
796	6.2 Load Hoisting	17
797	6.3 Nop Tunneling	18
798	7 Related Work	19
799	7.1 Verified Compilers	19
800	7.2 Relational Separation Logics	20
801	References	21
802	A Coinduction Example: Stream Bisimilarity	24
803	B Free Simulation Example	24

A Coinduction Example: Stream Bisimilarity

To illustrate how coinductive reasoning operates in practice, consider the *bisimilarity* relation $\approx$ on streams of natural numbers, $\text{Stream}(\mathbb{N})$, defined coinductively by the following inference rule:

$$\frac{n_1 = n_2 \quad s_1 \approx s_2}{\text{scons}(n_1, s_1) \approx \text{scons}(n_2, s_2)} \text{Bisim-Step}$$

Let us prove that this relation is reflexive:

$$\forall s \in \text{Stream}(\mathbb{N}), s \approx s.$$

The proof proceeds by coinduction on the *bisimilarity* relation:

1. Applying the coinduction principle provides the coinductive hypothesis:

$$\text{ColH} : \forall s' \in \text{Stream}(\mathbb{N}), s' \approx s',$$

with the restriction that ColH may only be applied under a constructor of the coinductive relation.

2. We decompose the stream s into a head $n \in \mathbb{N}$ and a tail $s' \in \text{Stream}(\mathbb{N})$, so that $s = \text{scons}(n, s')$. The goal becomes:

$$\text{scons}(n, s') \approx \text{scons}(n, s').$$

3. We apply the coinductive rule Bisim-Step. The first premise $n = n$ holds trivially, leaving the second premise $s' \approx s'$ to be established.
4. Because the constructor Bisim-Step has been applied, we can now legitimately invoke ColH. This immediately concludes the proof.

B Free Simulation Example

To illustrate how the free simulation operates in practice, we consider a small imperative language with commands $\text{skip} \mid c_1; c_2 \mid \text{while } b \{c\} \mid \text{ret } v$, where $\text{ret } v$ is a terminal configuration. The small-step reduction relation $c \rightarrow c'$ is defined by:

$$\begin{array}{ll} \text{skip}; c \rightarrow c & \\ \text{while } b \{c\}; k \rightarrow c; \text{while } b \{c\}; k & (\text{if } b = \text{true}) \\ \text{while } b \{c\}; k \rightarrow k & (\text{if } b = \text{false}) \end{array}$$

We instantiate the index domains with the natural numbers, $I = J = (\mathbb{N}, <)$, and define the target and source programs:

$$\begin{array}{l} P_t := \text{while } (b) \{ \text{skip} \}; \text{ret } () \\ P_s := \text{while } (b) \{ \text{skip}; \text{skip}; \text{skip} \}; \text{ret } () \end{array}$$

We wish to prove that $P_t \preceq_0 P_s \{=\}$. We proceed by coinduction on the free simulation. This gives us the following coinductive hypothesis:

$$\forall j > 0, \forall i > 0, \quad P_t \preceq_{j \sim i} P_s \{=\}.$$

We then perform case analysis on the boolean condition b :

- If $b = \text{false}$, both programs exit their loops in a single step: $P_t \rightarrow \text{ret } ()$ and $P_s \rightarrow \text{ret } ()$. Applying FTARGETSTEPS and FSOURCESTEPS, both configurations reach $\text{ret } ()$, which are final values returning (). The goal is thus closed immediately by FRELATED.
- If $b = \text{true}$, using FTARGETSTEPS the target loop unrolls: $P_t \rightarrow \text{skip}; P_t$, and the target index is updated to $j = 1$. Again with FTARGETSTEPS, the target reduces: $\text{skip}; P_t \rightarrow P_t$, and the target index is increased to $j = 2$. The simulation goal becomes $P_t \preceq_{2 \sim 0} P_s \{=\}$.

The source loop then executes four steps: first, $P_s \rightarrow \text{skip}; \text{skip}; \text{skip}; P_s$, updating the source index to $i = 1$ via FSOURCESTEPS ; next, three successive applications of FSOURCESTEPS consume the three `skip` statements, yielding index $i = 4$ and state P_s . The simulation goal becomes $P_t \lesssim_4 P_s \{=\}$.

Having accumulated positive progress indices ($0 < 2$ and $0 < 4$), we can apply the coinductive hypothesis to conclude the proof.

The complete derivation tree for the loop iteration case ($b = \text{true}$) is displayed in fig. 13.

$$\begin{array}{c}
 \frac{0 < 2 \quad 0 < 4}{P_t \lesssim_4 P_s \{=\}} \text{CIH} \\
 \frac{\frac{P_t \lesssim_1 \text{skip}; \text{skip}; \text{skip}; P_s \{=\}}{P_t \lesssim_0 P_s \{=\}} \text{FSOURCESTEPS } (\times 3)}{\text{skip}; P_t \lesssim_0 P_s \{=\}} \text{FSOURCESTEPS} \\
 \frac{\text{skip}; P_t \lesssim_0 P_s \{=\}}{P_t \lesssim_0 P_s \{=\}} \text{FTARGETSTEPS} \\
 \frac{P_t \lesssim_0 P_s \{=\}}{P_t \lesssim_0 P_s \{=\}} \text{FTARGETSTEPS}
 \end{array}$$

Figure 13: Derivation tree for the loop iteration case ($b = \text{true}$).

As illustrated by this example, the source and target programs can execute arbitrary numbers of asynchronous steps, at whatever pace is convenient, until matching invariants are reached.